Masir · مسیر

An AI career path assistant with resume-based job matching for Persian speakers — technical report

Rahnama College ML Bootcamp Hackathon · RFP RC-MLB-HACK-2026-01

Team: Sara (jobs), Hemo (taxonomy & graph), Amin (dialogue), Javad (resume parsing), Mahdi (learning resources & integration)

29 October 2026

1. Summary

Masir turns either a short Persian conversation (at most six questions) or an uploaded resume into one validated profile, and from that profile computes two things: a **learning roadmap** derived from an explicit skill graph, and a **ranked list of ten real job postings** with the matched and missing skills for each. The core design choice is to keep the language model on the edges — understanding free text — and to make every result the user sees (roadmap order, learning links, scores, explanations) come from deterministic code over curated data.

25	145	777	143	0.774	72
roles, 5 domains	concepts, 188 prerequisite edges	learning links, all checked	job postings, 3 sources	mean nDCG@10 (baseline 0.637)	automated tests passing

2. Architecture

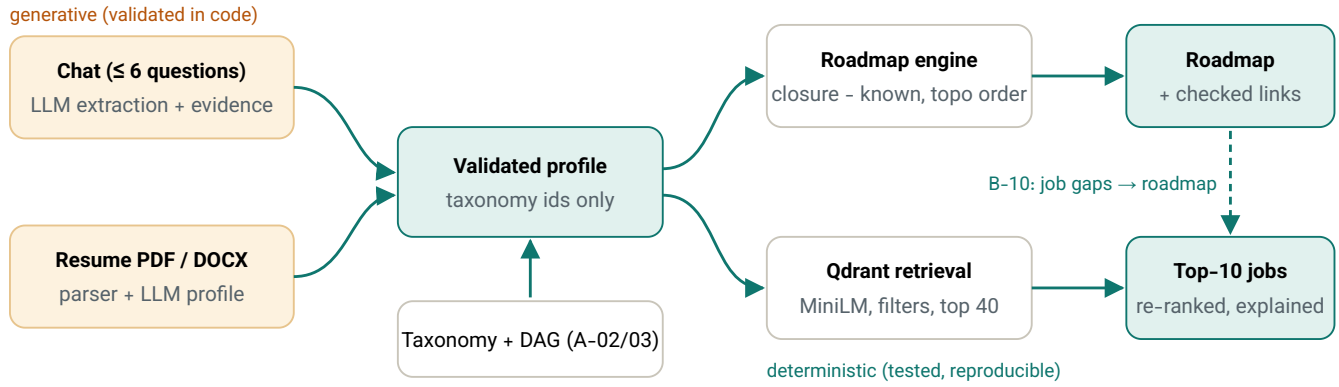


Figure 1. Data flow. Generative steps (amber) only produce structured data that is re-validated against the taxonomy; everything shown to the user is computed deterministically (teal).

The system is a FastAPI service (`server.py`) with a dependency-free right-to-left web client (`static/`). `docker compose up` starts it together with a Qdrant vector database (NF-07); without Docker, `run.cmd` / `run.sh` create a virtual environment and start the server, and Qdrant falls back to in-process mode. The language model is any OpenAI-compatible or Gemini-native gateway configured in `.env`; we tested on Metis with `gemini-3.1-flash-lite`. With no key, an offline rule-based extractor keeps the whole product usable.

3. Skill taxonomy and roadmap (Module A)

Roles are grouped in five domains (software, data & AI, finance & accounting, marketing & sales, management & HR), five roles each, covering every role the RFP names. Each role lists concepts per level; levels are incremental, so a mid-level target includes junior concepts. Concepts carry prerequisite edges forming a DAG, checked for cycles and dangling references at build time (A-03), and 866 hand-curated Persian/English aliases, kept unambiguous by the build script, which are how free text is mapped to concept ids.

The roadmap is computed, not generated (A-05): the prerequisite closure of the target concepts, minus the concepts the user has demonstrably used, in a deterministic topological order that puts junior-level foundations first. Each step lists up to three learning links drawn only from a catalogue of 777 resources (Maktabkhooneh, Aparat, YouTube, Coursera) collected per concept and HTTP-verified at build time (A-07); Persian links reachable from Iran are listed first. The Persian summary is generated from the same graph object, so text and graph cannot disagree (A-09).

Conversation and guardrails

- A state machine asks for target role, target level, education, experience and current level, and practical skills — at most six questions — with tap-to-answer options in the UI (A-01).
- The model returns JSON validated by Pydantic. Every claimed skill needs a verbatim quote from the user's message; negated or "only heard of it" skills are rejected, and unknown roles or concepts are discarded rather than shown (A-04).
- Corrections such as «من اکسل بلد نیستم» update the profile and regenerate the roadmap (A-08).
- To keep prompts small, only the target role's concepts and those named in the message are sent; measured cost was about 1.7k tokens and 1.4–1.6 s per turn.

4. Resume matching (Module B)

Resumes (PDF or DOCX, Persian or English) are parsed in memory with Persian normalisation of presentation forms, digits and letter variants (B-01). The structured profile — skills as taxonomy ids, years of experience, education, previous roles and location — has a documented JSON schema (`docs/resume_profile.schema.json`, B-02); the model's output is re-validated and merged with deterministic alias matches. In the integrated workspace a resume pre-fills the conversation profile, suggests the best-fitting roles and leaves only the missing questions.

The 143 postings come from Jobinja, e-estekhdam and Jobvision, normalised to one schema with de-duplication keys (B-03, B-09). A posting's skills are the taxonomy concepts found in its text plus collector tags plausible for its role.

Ranking (B-04 – B-07)

1. **Retrieval:** postings embedded once with multilingual MiniLM into Qdrant; the resume profile is embedded as a query; city/seniority/remote filters run inside Qdrant (B-08); top 40.
2. **Re-ranking** with four transparent features (below). Knowing a concept implies its prerequisites (Django ⇒ Python).

Feature	Weight
Coverage of the posting's required concepts (smoothed)	0.40
Fit to the posting's role (share of role concepts held)	0.25
Text similarity: mean of dense (Qdrant) and char n-gram TF-IDF	0.25
Seniority fit (years vs posting level)	0.10

Each result shows the score, a Persian explanation built from these numbers, matched and missing skills, and the link to the original ad (B-05, B-06). "Roadmap for this job" sends the missing concepts to the roadmap engine (B-10).

5. Evaluation (Section 10.2)

Three synthetic resumes mirror the RFP scenarios (a junior data analyst, a backend developer with two years of Python, an accountant with three years). Each is ranked against its own pool of 26 postings — the union of both rankers' top 10 plus same-domain and random postings — labelled 2 (strong fit), 1 (could realistically apply) or 0. Precision@5 counts grade ≥ 1; nDCG@10 uses gain 2^g-1. Profiles come from the deterministic extractor, so the numbers are reproducible with `python eval/evaluate_ranking.py`.

Resume	Keyword-overlap baseline P@5 / nDCG@10	Hybrid, no dense similarity P@5 / nDCG@10	RAG: Qdrant + hybrid (used) P@5 / nDCG@10
Junior data analyst	0.80 / 0.685	0.80 / 0.810	1.00 / 0.808
Backend developer (Python)	0.20 / 0.413	0.40 / 0.598	0.20 / 0.721
Accountant	1.00 / 0.814	1.00 / 0.793	1.00 / 0.794
Mean	0.67 / 0.637	0.73 / 0.733	**0.73 / 0.774**

The Qdrant retrieval stage alone returns 100 %, 100 % and 90 % of the relevant judged postings in its top 40 of all 143. The backend resume is hardest because the dataset has only one Python backend posting; most backend ads require .NET, Node.js or Java, which our labels treat as not relevant.

Caveats. Labels are a first draft by one team member and should be reviewed. With three resumes the gap between the two hybrid variants is within noise; the gain over the keyword baseline is the robust finding. An earlier run scored higher (0.841) before we removed wrong collector tags — for example web-development skills on a tax-accounting ad, which leaked HTML and JavaScript into that ad's roadmap. We kept the correction and report the lower numbers.

6. Responsible AI, engineering and cost

- **Grounding:** only taxonomy ids reach the profile; links come only from the verified catalogue; summaries and explanations are templated from computed data.
- **Privacy (NF-03):** a notice must be accepted before the first upload; resumes and chats stay in process memory, are never written to disk or logs, and expire after two hours or on "start over".
- **Secrets (NF-04):** keys live only in a git-ignored `.env`, read on the server.
- **Robustness (NF-06):** timeouts, malformed JSON, refusals and budget overruns return friendly Persian messages and keep the previous state.
- **Cost (NF-05):** per-session tokens, calls, latency and (with prices set) USD cost are tracked and shown; a session token budget is enforced.
- **Data collection (NF-08):** the resource collector is rate-limited and identifies itself; the job collector's code is to be added to the repository by its owner.

7. How it was built

The team split the work into five parallel tracks around shared contracts: `categories.json` defined the 25 `role_id`s; the learning-resource collector produced a draft concept list that the taxonomy track adopted, so resources, taxonomy and job tags all use the same concept ids. Integration then unified the taxonomy format between the graph engine and the dialogue guardrail (with cumulative levels so both compute identical closures), replaced the dialogue's reference graph engine with an adapter over the DAG engine, rebuilt Module B on the job data, added Qdrant retrieval and the evaluation harness, and replaced an early Streamlit prototype with a purpose-built RTL interface in which the conversation and the resume share one profile. Automated tests (72) cover the taxonomy, the dialogue guardrails, both RFP scenarios, ranking, the HTTP API and the served frontend.

Disclosure (RFP §6.2): an AI coding assistant (Claude Code) was used for parts of the implementation, integration, evaluation harness and these documents. Third-party components: FastAPI, Uvicorn, Pydantic, httpx, NetworkX, scikit-learn, NumPy, pdfplumber, python-docx, python-bidi, qdrant-client, FastEmbed, the Qdrant Docker image and the Vazirmatn font (SIL OFL).

8. Limitations and next steps

- More labelled resumes and a second labeller, to make the ranking comparison statistically meaningful.
- Add programming-language concepts (Java, C#/.NET, PHP) so stack mismatches are visible to the concept features, not only to text similarity.
- Commit the job collector and schedule incremental refreshes (B-11); re-verify learning links before each demo.
- Persian speech input and output (A-10) and roadmap export as an image or PDF (A-11).